FX Pricing Engine — Interview Guide

Oct 4, 2026 · @sreekar

1. Elevator pitch

I built a C++17 dynamic FX pricing engine that learns how customers react to price and quotes each one a profit-maximising spread. In simulation it earned 5.2% more profit than a tuned static rate sheet, and 7.8% more after its learning period, while cutting hedging cost by 28%.

30-second version (say this first):

> "Banks usually price retail and corporate FX from a static rate sheet: one spread per customer segment and currency pair. That leaves money on the table, because a $3,000 retail trade and a $600k corporate trade react to price very differently. I built a C++ engine that simulates an FX desk, learns each customer's probability of accepting a quote with a logistic regression I wrote from scratch, and picks the spread that maximises expected profit. It also skews prices to reduce the bank's open currency position. Across 10 simulated markets it beat a tuned rate sheet by 5.2% on profit, every single market improved, and hedging cost fell 28%."

2-minute version (if they say "tell me more"):

1. **Problem.** Static rate sheets ignore trade size, loyalty, volatility and the bank's inventory. Price too wide and you lose the trade; too tight and you give away margin.
2. **Simulated world.** 60 days × 500 quote requests per day. Three segments (Retail, HNI, Corporate), five pairs (USD, EUR, GBP, AED, SGD against INR). Each customer has a hidden maximum spread they will pay.
3. **Learning.** Every quote and accept/reject is logged. Each night a 30-feature logistic regression is refitted with Newton's method (IRLS). The first 8 days explore with random prices, then 3% of quotes keep exploring.
4. **Pricing.** For each request the engine tries 166 candidate spreads and picks the one that maximises P(accept) × (revenue − inventory cost).
5. **Fair test.** All four strategies replay the identical customers (common random numbers). The baseline sheet is tuned on a different random seed. Results are averaged over 10 seeds.

6. **Results.** +5.2% ± 1.0% profit vs the tuned sheet (+7.8% after warm-up), positive in all 10 markets, 28% lower hedging cost, 15% smaller average open position. Shown in an HTML/SVG dashboard.

One-line resume bullet: Built a C++17 FX pricing engine with a customer simulator, from-scratch logistic regression (Newton/IRLS) and an inventory-aware spread optimiser; improved simulated profit 5.2% (7.8% after warm-up) over a tuned static rate sheet across 10 seeds and cut hedging cost 28%.

Always say **"simulated"**. The numbers come from your own assumptions, not real bank data.

2. Problem and motivation

A static rate sheet charges everyone in a segment the same spread, so it overcharges price-sensitive customers (who walk away) and undercharges insensitive ones (margin lost).

Key terms to use confidently:

Term	Meaning in this project
Mid rate	The fair market price of a currency pair, e.g. USD/INR = 83.00
Spread	The bank's markup over mid, in basis points (bps). 1 bp = 0.01%. A 30 bp spread on $100k earns $300
Rate sheet	A table of spreads: one per (segment, pair) cell, 3 × 5 = 15 cells
Segment	Retail (60% of requests, median $3k), HNI (30%, $60k), Corporate (10%, $600k)
Acceptance / hit rate	Share of quotes the customer accepts
Inventory / open position	Net foreign currency the bank holds after customer trades; it carries price risk
Hedging	Offsetting a position in the interbank market, which costs a few bps
PnL	Profit and loss

The core trade-off. Expected profit from one quote = P(accept | spread) × spread × notional. A wider spread earns more per trade but wins fewer trades. The best spread is different for each customer, so one number per cell cannot be optimal.

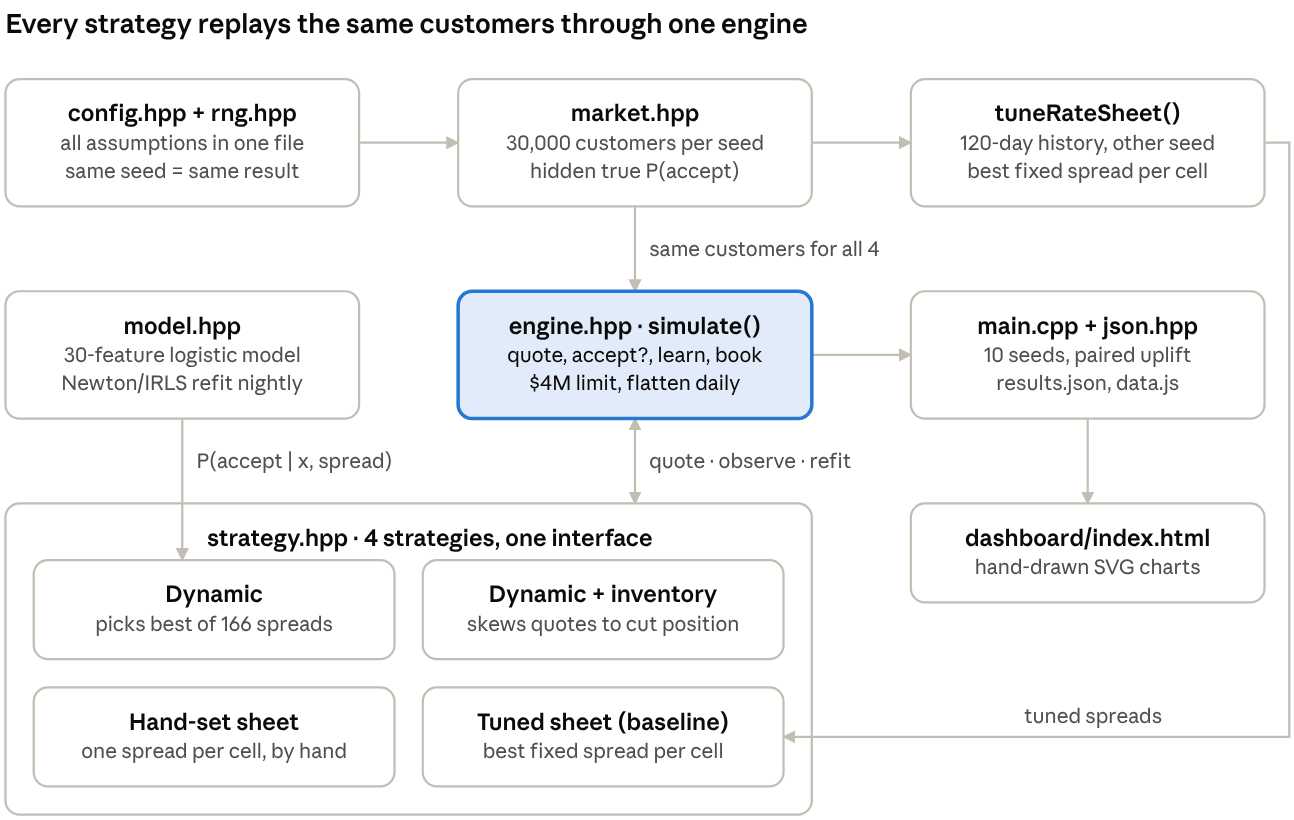
What a static sheet ignores:

- **Trade size.** Bigger trades shop around more, so they tolerate less spread.
- **Loyalty.** Long-standing customers tolerate more.
- **Volatility.** In nervous markets customers shop around less.
- **Side.** In this sim, customers selling FX are about 10% pickier.
- **The bank's own inventory.** If the bank is already short USD, a customer *selling* USD is worth more to the bank than one buying.

Why it matters to a bank (say this if the interviewer is from a bank): FX conversion is a high-volume, low-margin business. Small spread improvements across millions of trades add up, and inventory-aware pricing lowers hedging cost and market risk at the same time.

3. Architecture

The project is eight header-only C++17 modules with no external libraries. Each has one job, and the engine talks to every pricing strategy through one abstract interface.



project architecture · 8 source files, 4 strategies

market.hpp builds one stream of customers per seed. engine.hpp replays that same stream against each of the four strategies, and the dynamic ones ask model.hpp for P(accept) and retrain it every night. main.cpp averages the results over 10 seeds and writes the files the dashboard reads.

How to describe it: "I separated *the world* (market) from *the decision maker* (strategy) and *the accountant* (engine). Because of that, the same engine fairly scores any pricing idea, and adding a new strategy touches one file."

The README lists src/ and tests/ folders, but the files sit in one folder. Know this in case someone tries `make`.

4. The simulated world (market.hpp, config.hpp)

Each random seed generates one complete market of 60 days × 500 quote requests = 30,000 customers, generated once and replayed identically against every strategy.

Segments (config.hpp):

Segment	Share of requests	Median trade	Reservation spread	Width	Size elasticity	Hand-set sheet
Retail	60%	$3,000	110 bps	18 bps	0.15	85 bps
HNI	30%	$60,000	35 bps	6 bps	0.25	27 bps
Corporate	10%	$600,000	9 bps	1.6 bps	0.35	7 bps

Currency pairs:

Pair	Share	Tolerance factor	Hedge cost	Daily vol
USD/INR	45%	1.00	1.5 bps	0.30%
EUR/INR	20%	1.20	3.0 bps	0.50%
GBP/INR	12%	1.30	4.0 bps	0.55%
AED/INR	13%	0.90	2.0 bps	0.30%
SGD/INR	10%	1.25	4.5 bps	0.45%

The tolerance factor scales every spread for that pair: less liquid pairs carry wider spreads.

One quote request (struct Trade) holds: day, segment, pair, side (+1 = customer buys foreign currency, so the bank goes short), notional (log-normal around the segment median, clamped to $200–$5M), loyalty (uniform 0–1), the day's volatility regime, a hidden shock `latent` ~ N(0,1), and a pre-drawn uniform `u`.

How the market evolves each day:

- **Volatility regime:** a mean-reverting log process, logVol = 0.85 × logVol + 0.20 × N(0,1), then vol = exp(logVol) clamped to [0.6, 2.5]. 1.0 = a normal day.
- **Price move per pair:** dailyVol × vol × N(0,1). This drives mark-to-market (MTM) PnL.
- **One-sided flow:** each pair gets a daily buy probability 0.5 + bias, with bias ~ N(0, 0.12) clamped to ±0.35. Some days most customers buy USD, so inventory builds up.

Hidden customer behaviour (the ground truth). A customer accepts if u < P(accept), where:

```
P(\text{accept}) = \sigma\left(\frac{R - s}{w}\right), \qquad \sigma(z) =
\frac{1}{1 + e^{-z}}
```

s is the quoted spread, w is the width (segment width × pair tolerance) and R is the customer's reservation spread, the most they will pay:

```
R = R_{seg} \cdot \text{tol}_{pair} \cdot \left(\frac{N}
{N_{median}}\right)^{-\epsilon} \cdot (1 + 0.25(v - 1)) \cdot \text{side}
\cdot (0.75 + 0.5L) \cdot e^{0.2z}
```

- Size: bigger trades tolerate less (elasticity ε = 0.15 / 0.25 / 0.35).
- Volatility v: in nervous markets customers tolerate more.
- Side factor: 1.0 if buying, 0.9 if selling.
- Loyalty L: from 0.75× (L = 0) to 1.25× (L = 1).
- e^{0.2z}: a personal shock the bank can never see. It sets the floor on how well anyone can predict.

Explain it like this: "R is the highest price this customer would pay. When my spread equals R it's a coin flip. Width w controls how sharply acceptance falls as I go above R: corporates have a tiny width (1.6 bps), so they're very price-sensitive."

Common random numbers. The stream is generated once per seed, including each customer's `u` and `latent`. Every strategy faces exactly the same customers. A customer who would accept 20 bps from strategy A also accepts 20 bps from strategy B. So the PnL difference comes from pricing decisions, not luck, and comparisons need far fewer seeds.

The "best possible" predictor. marginalAcceptProb averages P(accept) over the hidden shock using 9-point Gauss–Hermite quadrature. This is the best prediction anyone could make from observable data, and it is used as the oracle benchmark for the model's log-loss.

Deterministic RNG (rng.hpp). std::normal_distribution can give different numbers on different compilers. So I use only std::mt19937_64 (its output is fixed by the C++ standard) and build uniform, Box–Muller normal and categorical draws by hand. The same seed gives the same result on any machine.

5. The acceptance model (model.hpp)

The model is a 30-feature logistic regression, written from scratch, that predicts P(accept | customer, market, spread). It is refitted every night on all data so far using Newton's method (IRLS) with an L2 ridge penalty of 1.0.

```
P(\text{accept}) = \sigma(w \cdot x)
```

The 30 features:

Group	Count	Value	Why it's there
segment	3	one-hot	Base acceptance level per segment
pair	5	one-hot	Base level per pair
size × segment	3	log(notional / segment median)	Size effect differs by segment
vol × segment	3	vol − 1	Volatility effect per segment
loyalty × segment	3	loyalty − 0.5	Loyalty effect per segment
side	1	+1 / −1	Buyers vs sellers
spread × segment	3	sRel = spread / reference spread	Price sensitivity per segment
spread × pair	5	sRel	Price sensitivity per pair
spread × size, vol, side, loyalty	4	sRel × each	Price sensitivity shifts with context

Design details worth mentioning:

- **No separate intercept.** The segment one-hots always sum to 1, so they act as the intercept. The pair one-hots also sum to 1, so the two groups are perfectly collinear. The ridge penalty makes the problem well-posed anyway: it keeps the Hessian positive-definite.
- **Spread is normalised.** sRel = spread / reference spread puts Retail (≈85 bps) and Corporate (≈7 bps) on the same scale, so one set of weights works across segments.

- **The logit is linear in the spread.** Every spread feature has the form sRel × something. So for a given customer, logit = a + b × sRel, which matches the true model's shape σ((R − s)/w). The model is well-specified, apart from the hidden shock.
- **One function builds features and names.** emitFeatures takes a callback. The same code fills the numbers and lists the names for the dashboard, so they can never drift apart.
- **Centred inputs.** vol − 1 and loyalty − 0.5 keep features near zero, which makes the weights easier to read and the optimisation better conditioned.

How the fit works (IRLS / Newton). The loss is log-loss plus (ridge/2)·‖w‖². Each iteration:

```
g = \sum_i (p_i - y_i)\,x_i + \lambda w \qquad H = \sum_i p_i(1-p_i)\,x_i
x_i^\top + \lambda I \qquad w \leftarrow w - H^{-1}g
```

- H is symmetric positive-definite, so the step is solved with a hand-written **Cholesky decomposition** ($H = LL^T$, then forward and back substitution). It never forms H^{-1} explicitly.
- Only the lower triangle is accumulated, then mirrored: about half the work.
- Zero features are skipped (`if (s.x[i] == 0.0) continue`), since one-hots are mostly zero.
- **Warm start:** each night starts from yesterday's weights, so it usually converges in 1–3 iterations. Stops when the largest step < 1e-5, max 12 iterations.
- Predictions are clipped to [1e-6, 1 − 1e-6] so log-loss stays finite.
- Cost per night: $O(n \times 30^2)$ to build H plus $O(30^3)$ for Cholesky. With 30,000 samples this is milliseconds.

Why Newton instead of gradient descent? Logistic loss is convex, and Newton uses curvature, so it converges quadratically in a few steps with no learning rate to tune. With 30 features the 30 × 30 Hessian is cheap. Plain SGD converged too slowly and gave a model that was not price-sensitive enough, so it over-quoted.

Exploration (in strategy.hpp). A model only learns a price-response curve if it sees customers react to different prices.

- Days 1–8 (warm-up): random spread = reference × U(0.6, 1.5).
- After that: 3% of quotes (epsilon-greedy) use reference × U(0.7, 1.4).
- Without this, the engine would always quote near its own optimum, and the spread coefficient would be poorly identified.

Evidence the model works (results.json; log-loss averaged over 10 seeds, weights from seed 1):

- Day 1 log-loss = 0.693 = ln 2, because all weights start at zero (predicting 50%).
- By days 2–8 it is within about 0.01–0.03 of the best-possible (oracle) log-loss.
- On day 9 both the model's and the oracle's loss jump to about 0.6. That is expected: the optimiser now quotes near the profit-maximising point, where outcomes are genuinely more uncertain. The model tracks the oracle closely.
- The learned weights make economic sense. All spread weights are negative (higher price, fewer accepts). size × Corporate (−1.05) is the most negative size effect, matching the highest true elasticity (0.35). Loyalty weights are positive.

6. The four strategies (strategy.hpp)

All strategies share one interface, so the engine can replay any of them without knowing which it is. The dynamic engine wins by charging each customer what they will bear, not by quoting tighter.

The Strategy interface (virtual methods): quote(context, positions, day) returns a spread; observe(...) learns from the answer; endOfDay(day) retrains overnight; predictAccept(...) returns the model's probability, or NaN for static sheets.

Strategy	How it prices	Learns?	Uses inventory?
Static, hand-set	sheetBps × pair tolerance, fixed	No	No
Static, tuned (baseline)	Best fixed spread per cell from a separate 120-day history	Offline only	No
Dynamic	argmax over 166 spreads of P(accept) × revenue	Nightly refit	No
Dynamic + inventory-aware	argmax of P(accept) × (revenue − inventory cost)	Nightly refit	Yes

Hand-set vs tuned. Both are one fixed spread per (segment, pair) cell. Hand-set = the reference spread a person wrote down. Tuned = for each cell, multipliers 0.30× to 2.00× in 0.01 steps are replayed over a 120-day history from a different seed (9000 + seed), and the

most profitable one is kept. It's like a desk that has run A/B tests. The tuned sheet is the headline baseline because beating a badly set sheet proves nothing.

The dynamic optimiser. For each request (after warm-up, when not exploring):

$$s^* = \arg\max_{s} \; P(\text{accept} \mid x, s) \times \left(\frac{s}{10^4} \cdot N - C_{inv}\right)$$

- Candidate spreads: reference × 0.35 to 2.00 in 0.01 steps = 166 candidates.
- Why a grid and not calculus? It's simple and robust, needs no derivative of the inventory term, and stays inside safe bounds. 166 × 30 multiply-adds per quote is trivial.
- C_inv = 0 for plain dynamic pricing.

The inventory penalty (USD) for the inventory-aware version:

$$C_{inv} = \theta \cdot h \cdot (|q_{after}| - |q_{before}|) + \tfrac{1}{2}\gamma\sigma^2 (q_{after}^2 - q_{before}^2)$$

- q is the bank's open position in that pair. A customer buying foreign currency makes the bank shorter: q_after = q_before − side × notional.
- **Hedging term:** θ = 0.6 is the share of new one-sided flow expected to end up hedged, at hedge rate h = hedgeBps × vol.
- **Risk term:** γ = 3e-6 is risk aversion and σ = daily vol × vol regime. This is the classic mean–variance (Avellaneda–Stoikov style) inventory penalty.
- **Sign matters:** if the trade *reduces* the position, the penalty is negative. The engine then quotes tighter to win offsetting flow, and quotes wider for trades that make things worse. This is called **skewing**.
- θ and γ were chosen on seeds 101–104 and reported on seeds 1–10, so they are not tuned on the test data.

What the engine actually learned to do (steady-state averages):

Segment	Tuned avg spread	Tuned accept %	Dynamic avg spread	Dynamic accept %
Retail	84.1 bps	75.8%	94.8 bps	71.4%
HNI	26.9 bps	74.5%	30.7 bps	70.0%
Corporate	6.45 bps	76.0%	8.31 bps	67.0%

The dynamic engine quotes *wider on average* and wins *fewer* trades, yet earns more. It charges more to customers who will pay (small, loyal, calm market) and less to those who won't (large, new). The tuned sheet can only pick one compromise number per cell. **This is price discrimination by willingness to pay, and it's the single most important insight to say in the interview.**

7. Engine, experiment design and results

Across 10 seeds, dynamic + inventory-aware pricing earned $2.55M per market vs $2.42M for the tuned sheet (+5.2% ± 1.0%), and improved profit in all 10 markets.

The per-quote loop in simulate() (engine.hpp):

1. The strategy quotes a spread, given the context and the bank's current positions.
2. The customer answers: accepted = u < true P(accept), using the hidden shock.
3. Diagnostics: the log-loss of the strategy's prediction vs the oracle is recorded *before* it learns from the outcome, so it's an honest out-of-sample score.
4. The strategy observes the outcome (stores a training sample).
5. Bookkeeping. If accepted: revenue += spread/10,000 × notional, and position −= side × notional.
6. **Hard limit:** if |position| > $4M in a pair, the engine hedges back to $2M (50% of the limit) and pays hedgeBps × vol on the amount.

End of each day: endOfDay() triggers the overnight refit. Then the engine records total |inventory|, books MTM = position × the day's price move, and **flattens every position** at the hedge cost.

PnL definitions:

- **Core PnL = spread revenue − hedge cost.** This is the headline number.
- **MTM PnL** = position × price move. Its expected value is zero, so it's pure noise for comparison. It's reported separately, and its daily standard deviation is used as a **risk** measure.
- **Steady PnL** = core PnL after the 8 warm-up days. It shows performance once learning is done.

Experiment design (main.cpp): for each seed 1–10:

1. Generate the evaluation market (60 days).

2. Generate an independent 120-day history (seed 9000 + s) and tune the static sheet on it.
3. Replay the *same* market against all four strategies.
4. Compute each strategy's uplift vs the tuned sheet **within the same seed**, then report mean ± std and min/max across seeds.

Comparing within each seed is a **paired comparison**. The market-level noise cancels, which is why a std of about 1% is achievable with only 10 seeds.

Results (10 seeds, 60 days × 500 quotes):

Strategy	Core PnL / market	vs tuned sheet	Accept rate	Avg open position	Hedge cost	Daily MTM std
Hand-set sheet	$2.40M	−1.2%	68.9%	$7.25M	$171k	$17.9k
Tuned sheet (baseline)	$2.42M	—	75.2%	$7.70M	$198k	$18.9k
Dynamic	$2.55M	+5.1% ± 0.8	69.4%	$7.63M	$202k	$18.5k
Dynamic + inventory	$2.55M	+5.2% ± 1.0	69.0%	$6.50M	$145k	$15.2k

How to read it:

- **After warm-up** the uplift is +7.8% ± 0.9. The overall figure is lower because the 8 exploration days cost money.
- **Every seed improved:** uplift ranged from +3.7% to +7.0% across the 10 markets.
- **Inventory awareness** doesn't change profit (+5.2% vs +5.1%) but cuts hedging cost by 28% ($202k → $145k), average open position by 15% and daily MTM risk by 18%. Same money, less risk: a risk manager's favourite result.
- **The tuned sheet wins more trades** (75% vs 69%) yet earns less. It buys volume with tight spreads, while the dynamic engine prices each customer.

Dashboard (dashboard/index.html). main.cpp writes results.json and data.js (`window.FXDATA = …`). Loading a .js file lets the page open straight from disk without a web server, because browsers block fetch() of local files. The page is plain HTML/CSS/JS with hand-drawn SVG charts showing cumulative PnL, inventory, model log-loss vs oracle, learned vs true acceptance curves per segment, segment stats and the learned weights. JSON is written by a tiny hand-rolled writer (json.hpp), so the project has **zero external dependencies**.

Tests (test_core.cpp): RNG determinism, Gauss–Hermite quadrature accuracy, stream generation, the model recovering the true behaviour, accounting identities, and a check that dynamic pricing beats the hand-set sheet.

8. Limitations and future work

Raise the limitations yourself before the interviewer does. It signals maturity, and every one has a clear next step.

Honest caveats:

- **The world is my assumption.** Customer behaviour follows the exact sigmoid shape the model assumes, so the model is close to well-specified. Real customers would be messier. The 5.2% belongs to this simulator, not to a real desk.
- **The tuned baseline peeks at ground truth.** tuneRateSheet scores each candidate spread with the customers' true reservation and width on the history stream. A real desk only sees accept/reject at the prices it actually quoted. So the baseline is, if anything, *stronger* than reality, which makes the uplift conservative.
- **No competitors.** Real customers compare quotes from several banks, and acceptance depends on rival prices.
- **Customers don't repeat.** Loyalty is random, not earned. Overcharging today has no future cost in the sim, whereas in reality customers churn.
- **Fairness and regulation.** Charging different customers different prices for the same product raises conduct and fairness questions (for example RBI and FX Global Code expectations, and customer-treatment rules). A real deployment needs spread caps, explainability and monitoring.
- **Memory and refit grow with time.** All samples are stored and every refit passes over all of them: $O(n)$ memory and $O(n \cdot 30^2)$ per night. Fine for 30k samples, not for years of history. Fixes: a rolling window, exponential decay weights, or sufficient statistics.
- **No uncertainty-aware exploration.** Exploration is a flat 3% random. It doesn't target where the model is unsure.
- **Flatten daily is a simplification.** Real desks carry positions overnight and hedge more cleverly.

What I'd do next:

1. Thompson sampling or UCB (contextual bandit) to explore only where the model is uncertain.

2. Compare against gradient-boosted trees behind the same Strategy interface, with calibration checks.
3. Repeat customers with per-customer history and a churn model, so pricing optimises lifetime value.
4. Competitor quotes as features.
5. Serve the engine as a low-latency quote API and A/B test it on live traffic with guardrails: spread caps and fairness limits.
6. Fix the README's src/ and tests/ paths to match the actual folder, so `make` works out of the box.

9. Interview Q&A

These are the questions you're most likely to get, with answers you can say out loud. Keep each answer to 30–60 seconds and offer to go deeper.

Project overview

Q: Walk me through your project. Use the 2-minute version from section 1: problem → simulated world → learning → optimiser → fair test → results. End with the key insight: it wins by pricing each customer to their willingness to pay, not by quoting tighter.

Q: Why did you build this? I wanted a project that joins machine learning, optimisation and real banking economics, not just a model with an accuracy score. FX pricing is a good fit: the decision (the spread) directly changes the data you get back (accept/reject), so you have to think about exploration, not just fitting.

Q: What was the hardest part? Making the comparison fair. Beating a hand-set sheet proves nothing, so I built a tuned baseline fitted on a different seed, common random numbers so every strategy sees the same customers, and multiple seeds with paired uplifts. The second hardest part was getting the model price-sensitive enough: SGD under-fit the spread effect and the engine over-quoted, which Newton's method fixed.

Q: Is the data real? No, everything is simulated, and I say that explicitly. The advantage is that I know the ground truth, so I can measure how close the model gets to the best possible prediction. That's impossible with real data.

Machine learning

Q: Why logistic regression, not XGBoost or a neural net? The outcome is binary and I need *calibrated* probabilities, because they go straight into an expected-profit formula. Logistic regression is calibrated by construction, fast to refit nightly, interpretable, and its logit is linear in the spread, which matches the true behaviour's shape. It already reaches near the oracle log-loss, so a more complex model has little room to add value here. On real data I'd benchmark trees too.

Q: Explain Newton's method / IRLS. Newton's method uses both the gradient g and the curvature (Hessian H) of the loss, and steps $w \leftarrow w - H^{-1}g$. For logistic regression $H = \Sigma\, p(1-p)xx^{T} + \lambda I$. Each step is equivalent to a weighted least-squares problem with weights $p(1-p)$, hence "iteratively reweighted least squares". It converges quadratically, with no learning rate.

Q: Why Cholesky? H is symmetric positive-definite (guaranteed by the ridge term). Cholesky ($H = LL^{T}$) is the fastest stable way to solve such a system, about half the cost of LU. You never invert the matrix: just forward then back substitution.

Q: What is the ridge term doing? Three things: it prevents over-fitting early when data is scarce, it makes H invertible even though my segment and pair one-hots are perfectly collinear (both sum to 1), and it shrinks weights toward zero for stability.

Q: Why is there no intercept? The segment one-hots sum to 1 for every row, so together they already act as the intercept. Adding another would be redundant.

Q: How do you know the model is good? Three checks. Log-loss vs the oracle (the best possible predictor that knows the true behaviour but not the hidden shock): the gap closes within days. Learned vs true acceptance curves on the dashboard. And the weights' signs make economic sense: spread is negative, loyalty positive, and the size effect strongest for corporates.

Q: What's log-loss and why use it? Log-loss = $-[y \log p + (1-y) \log(1-p)]$. It punishes confident wrong predictions heavily. I care about calibrated probabilities, not just right/wrong classification, so accuracy would be the wrong metric.

Q: Why does log-loss jump on day 9? Warm-up ends and the engine starts quoting at its profit-maximising spread, where acceptance is genuinely uncertain (probabilities nearer 50–70%). The oracle's loss jumps by the same amount, so it's the data getting harder, not the model getting worse.

Q: What is the exploration–exploitation trade-off here? If I always quote my current best price, I only see outcomes near that price and can't learn the slope of the demand curve. So I explore: 8 days of random prices, then 3% epsilon-greedy. Exploration costs money now (visible in the early PnL) and buys a better model later. It's a contextual bandit problem.

Q: Isn't refitting on all data every night slow? It's $O(n \cdot 30^2)$ per night, which is milliseconds for 30k samples. At real scale I'd keep running sufficient statistics, use a rolling window or decay old data, which also handles customer behaviour drifting over time.

Pricing, finance and risk

Q: What is a basis point and a spread? 1 bp = 0.01%. The spread is the markup over the mid rate. 30 bps on a $100,000 trade earns $300.

Q: Why does the dynamic engine win while accepting fewer trades? It prices to each customer's willingness to pay. Small, loyal customers in calm markets tolerate more, so it charges them more. Large, price-sensitive ones get tighter quotes. The tuned sheet must use one compromise number per cell, so it over-tightens to buy volume (75% accept rate vs 69%) and earns less.

Q: Explain the inventory penalty. Two parts. A hedging term: the expected cost of eventually hedging any increase in |position|. And a risk term: $\frac{1}{2}\gamma\sigma^2$ times the change in position^2, the classic mean–variance penalty used in market-making models like Avellaneda–Stoikov. If a trade reduces my position the penalty is negative, so I quote tighter to attract it. That's called skewing.

Q: Inventory awareness didn't increase profit. Why include it? Profit stayed the same, but hedging cost fell 28%, average open position 15% and daily price risk 18%. Same return with less risk is a strictly better risk-adjusted outcome. A risk desk would value that.

Q: How did you choose θ and γ? Isn't that overfitting? I tuned them on separate seeds (101–104) and reported results on seeds 1–10, the same idea as a validation set vs a test set.

Q: What is MTM PnL and why exclude it from the headline? Mark-to-market = position × price move. Its expected value is zero, since prices are a random walk in the sim, so including it just adds noise. I report it separately and use its standard deviation as a risk measure.

Q: What does the hard limit do? If any pair's position exceeds $4M, the engine immediately hedges back to $2M and pays the interbank cost. It's a simple risk limit, like a desk's position limit.

Experiment design and statistics

Q: What are common random numbers and why do they matter? Each customer's random draw for accepting (u) and their hidden shock are generated once and reused for every strategy. Differences in PnL then come from pricing, not luck. It's a variance-reduction technique, so I need far fewer simulations for a confident comparison.

Q: Is 10 seeds enough? Is the result significant? The uplifts are paired within seed: mean +5.2%, std 1.0%, and all 10 seeds positive. A rough t-statistic is 5.2 / (1.0/√10) ≈ 16, so it's very unlikely to be noise. For a publication I'd run more seeds and report a confidence interval.

Q: How do you make sure the tuned baseline isn't cheating, or that you aren't? The tuned sheet is fitted on a separate 120-day history with a different seed (9000 + s). The dynamic engine learns only from its own quotes during the test, with no access to the hidden truth. Diagnostics score predictions *before* the model sees each outcome.

Q: Why is the uplift higher after warm-up (7.8%) than overall (5.2%)? The first 8 days are random exploration, which costs money relative to a good fixed sheet. Once the model is trained the uplift is about 7.8%. The overall figure includes that learning cost, so it's the honest one to quote.

C++ and engineering

Q: Why C++? Pricing engines in banks need low latency and predictable performance. C++17 with no external libraries also forced me to implement the maths myself: the regression, Cholesky, the RNG and JSON. A full 10-seed run takes a few seconds.

Q: Why write your own RNG wrapper? std::normal_distribution isn't specified exactly by the standard, so the same seed gives different numbers on GCC vs MSVC. std::mt19937_64's output *is* specified, so I build uniform, Box–Muller normal and categorical draws on top of it. That gives bit-for-bit reproducibility across machines.

Q: How is the code structured? Header-only modules with one responsibility each: config (assumptions), rng, market (world and ground truth), model (ML), strategy (pricing), engine (accounting), json, and main (experiment). Strategies share an abstract Strategy base class with virtual quote/observe/endOfDay methods, so the engine is decoupled from pricing logic. Adding a new strategy needs no engine changes.

Q: What C++ features did you use? std::array for fixed-size vectors and matrices (stack-allocated, no heap churn), constexpr config tables, templates with lambdas (emitFeatures takes a callback so one function produces both values and names), pointer-to-member (meanSeries works on any RunResult series), virtual dispatch, and inline header-only functions.

Q: How would you make it production-ready? A quote service behind an API with latency budgets, model training moved offline with versioned weights, monitoring of calibration and acceptance drift, spread caps and fairness checks, logging every quote for audit, and a live A/B test with a kill switch.

Q: How did you test it? Unit tests (test_core.cpp) for RNG determinism, the quadrature accuracy, stream properties, the model recovering the true behaviour on synthetic data, accounting identities (revenue − hedge = core PnL), and an end-to-end check that dynamic pricing beats the sheet.

Behavioural, about this project

Q: What would you do differently? Model customers who come back over time, so overcharging today has a real cost tomorrow. And use smarter exploration (Thompson sampling) rather than flat epsilon-greedy.

Q: Tell me about a bug you hit. Suggested story, adapt to what really happened: the SGD-trained model was not price-sensitive enough, so the engine kept quoting too wide and lost trades. I diagnosed it by comparing learned vs true acceptance curves, switched to Newton/IRLS, and the curves lined up. *Only tell stories that actually happened to you.*

Q: What did you learn? That in decision problems the model's job is to support a decision, so calibration matters more than accuracy. That the evaluation design (baselines, paired comparisons, held-out tuning) is as important as the model. And that you should be honest about what a simulation can and can't prove.

Quick-fire numbers to memorise

Fact	Value
Days × quotes per day	60 × 500 = 30,000 per seed
Seeds	10 (test), 101–104 (tuning θ, γ)
Features	30

Fact	Value
Candidate spreads	166 (0.35× to 2.00× reference)
Warm-up · exploration	8 days · 3%
Uplift vs tuned sheet	+5.2% ± 1.0 overall, +7.8% ± 0.9 after warm-up
Uplift range across seeds	+3.7% to +7.0%
Hedging cost cut (inventory-aware)	28%
Open position cut · MTM risk cut	15% · 18%
Hard position limit	$4M per pair, hedge back to $2M